



Unraid Watcher

Documentation

Version 1.4.0

© 2026 Ray Munro

Licensed under the GNU General Public License v3.0 or later

Source code: <https://github.com/RayMunro/unraid-watcher>

Contents

1. [Overview](#)
2. [Getting started](#)
3. [User guide](#)
4. [Settings](#)
5. [Alerts and notifications](#)
6. [Controls and safety](#)
7. [Troubleshooting](#)
8. [Data sources](#)
9. [Architecture](#)
10. [Building and releasing](#)
11. [Changelog](#)

Unraid Watcher

A native macOS dashboard and control panel for your Unraid server, with a menu bar status item.

Unraid Watcher shows you what your server is doing (array health, disks, containers, VMs, shares, temperatures, network traffic, logs) and lets you act on it (start and stop containers and VMs, manage the array, run parity checks, create and clone VMs, manage shares) without opening the Unraid web interface.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Highlights

- **Multiple servers:** monitor and control as many Unraid servers as you like, with an all-servers overview and a switcher
- **Live overview** of CPU, memory, array state, Docker, VMs, and alerts, with a progress bar and percentage for any rebuild, parity build, or parity check
- **Disk health:** usage, temperatures, parity history, and SMART details per disk
- **Docker and VM control**, including VM creation, editing, cloning, and console access
- **Share management**, with a view of how any share is spread across your drives
- **Cache clean-up:** find files the mover left behind (verified identical copies already on the array) and empty folders on your cache pools, with a preview, confirmation, and a byte-for-byte re-check before any file is deleted
- **Deluge panel** (when the server runs Deluge): speeds, every torrent, and pause, resume, remove, and add controls
- **Performance and network** views: per-core load labelled by core type (performance, efficiency, low-power), NPU and GPU activity, disk I/O, fans, CPU temperatures, and traffic
- **macOS notifications** for hot disks, a hot CPU, SMART problems, and new Unraid alerts
- **Menu bar** summary that is always one click away, with an option to launch at login
- **Secure by default:** the API key and SSH password live in the macOS Keychain

Requirements

Item	Requirement
Mac	macOS 14 (Sonoma) or later
Unraid	7.2 or later (built-in API), or an older version with the Connect plugin
API key	Required. Viewer role to monitor, Admin role to use controls
SSH	Optional, but needed for temperatures, network traffic, SMART, logs, and the VM and share tools

Quick start

1. Build the app and open it (see [Building and releasing](#)), or install it from the DMG.
2. Open **Settings** (Cmd+,).
3. Enter your server address, for example `http://192.168.1.10`, and your API key.
4. Optionally turn on SSH and enter your SSH user and password.
5. Click **Save & Connect**.

The full walkthrough is in [Getting started](#).

Documentation

Guide	What it covers
Getting started	Creating an API key, enabling SSH, first connection
User guide	Every tab and what each control does
Settings	All preferences, defaults, and where data is stored
Alerts and notifications	What triggers a notification and how to tune it
Controls and safety	Destructive actions, confirmations, and permissions
Troubleshooting	Common errors and how to fix them
Data sources	Every API query, mutation, and SSH command the app uses
Architecture	How the code is organized
Building and releasing	Building the app, icon, and DMG installer
Changelog	Version history

All of these guides are also combined into one PDF: [Unraid-Watcher-Documentation.pdf](#). The installer includes a copy.

Privacy and security

- Unraid Watcher talks only to the server address you enter. It has no analytics, no telemetry, and no third-party services.
- Your API key and SSH password are stored in the macOS Keychain, never in plain text on disk.
- Destructive actions (stopping the array, rebooting, deleting a share, force-stopping a VM) always ask for confirmation first.

License

Unraid Watcher is free software: you can redistribute it and modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

It is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the [LICENSE](#) file for the full text.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Disclaimer

This is an independent project. It is not affiliated with, endorsed by, or sponsored by Lime Technology, Inc. Unraid is a trademark of Lime Technology, Inc. This software can change settings, stop services, and delete share and VM definitions on your server. Use it at your own risk and keep backups, as the license above makes clear.

Getting started

This guide takes you from a fresh install to a connected dashboard.

1. Prepare your Unraid server

Create an API key

Unraid Watcher reads data through Unraid's GraphQL API, which needs an API key.

1. Open the Unraid web interface.
2. Go to **Settings, Management Access, API Keys**.
3. Create a new key.
4. Choose a role:
 - **Viewer** is enough to monitor everything on the dashboard.
 - **Admin** is needed for actions that change things through the API: starting and stopping containers, the array, parity checks, VM power actions, and archiving notifications.
5. Copy the key. You will paste it into Unraid Watcher.

Unraid 7.2 and later include the API. On older versions, install the Unraid Connect plugin to get it.

Enable SSH (optional but recommended)

Some features read data or run commands directly on the server over SSH:

- CPU temperatures, fan speeds, network traffic, load, memory detail, disk I/O, and top processes
- SMART health details and self-tests
- The system log viewer and container logs
- Spinning disks up or down, the mover, reboot and shutdown
- VM details, creation, editing, cloning, media, and console access
- Share settings and share management
- The command console

To enable it, open **Settings, Management Access** in Unraid and turn on **Use SSH**. Make sure you know the root password, or set up key-based login.

Find your server address

Use the same address you type in a browser to open the Unraid web interface, for example `http://192.168.1.10` or `http://tower.local`. If you use HTTPS with a self-signed certificate, see [Settings](#).

2. Connect Unraid Watcher

1. Open Unraid Watcher.
2. Open **Settings** with Cmd+, (or the app menu) and stay on the **Servers** tab.
3. Click + to add a server (or select the one that is already there), then fill in:
 - **Name**: anything you like, for example `Prime`
 - **Server URL**: your server address
 - **API key**: the key you created
4. To use SSH features, turn on **Read temperatures & network over SSH** and fill in:
 - **SSH user**: usually `root`
 - **SSH password**: your server password, or leave it empty to use your Mac's SSH keys
5. Click **Save & Connect**.

macOS may ask for your login keychain password the first time the app reads or writes the saved API key or SSH password. Enter it and choose **Always Allow**. This is a normal macOS prompt, and the app never sees your keychain password.

Adding more servers

Repeat step 3 with another +. Each server has its own address, key, and SSH settings, and all of them are monitored at the same time. Switch between them with the picker at the top of the sidebar, or press Cmd+1, Cmd+2, and so on. See the [User guide](#) for the all-servers overview.

3. Check that it works

- The **Overview** tab should show your hostname, CPU, memory, and array state within a few seconds.
- A small timestamp ("Updated ...") appears in the toolbar after each successful refresh.
- Any problems appear as orange messages at the top of the page. Click **Copy all errors** to copy them. See [Troubleshooting](#) for what they mean.

SSH with keys instead of a password

If you prefer keys, leave the SSH password empty and run this once in Terminal:

```
ssh-copy-id root@your-server-address
```

Unraid Watcher uses your Mac's SSH keys when no password is set. With a password set, it still tries keys first and then falls back to the password.

Next steps

- Read the [User guide](#) for a tour of each tab.
- Tune thresholds in [Alerts and notifications](#).
- Review [Controls and safety](#) before using the power and array controls.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

User guide

Working with several servers

Every server you add in Settings is monitored in the background at the same time, so alerts keep working for all of them whichever one you are looking at.

- **Server picker:** the box at the top of the sidebar shows the server you are viewing, with a dot that is green when connected, red when it cannot be reached, and grey when it has not been set up. Click it to switch servers, or choose **Manage servers** to open Settings.
- **Keyboard:** the **Servers** menu in the menu bar lists your servers. Press Cmd+1 to Cmd+9 to jump to one, and Cmd+0 for the all-servers overview.
- **All servers overview:** choose **All servers** in the picker for one card per server showing its status, array state and usage, CPU and memory, Docker and VM counts, hottest disk, and unread alerts. Click a card to open that server. **Refresh all** updates every server at once.
- **Which server am I changing?** Everything in the tabs below applies to the server named in the picker and at the top of the window. Confirmations for the array, reboot, and shutdown controls repeat the server's name, so you can check before you confirm.

The rest of this guide describes the tabs for one server.

The window has a sidebar on the left and a detail area on the right. A footer at the bottom of the sidebar shows the app name and copyright. Click it to open the About window.

The **refresh button** in the toolbar updates everything immediately. The app also refreshes on its own every few seconds (see [Settings](#)).

Features marked **SSH** need SSH turned on in Settings. Features marked **Admin** need an API key with Admin rights.

Overview

A summary of the whole server:

- **Array operation:** appears only while the array is rebuilding a disk, building parity, running a parity check, or clearing a new disk. It shows what is running, a progress bar with the percentage on it, how much has been done out of the total, the speed, and about how long is left. Rebuilds and parity builds add a note on what they mean for your data, for example that a disk is being emulated from parity and the array cannot survive another disk failure. The percentage also appears beside Overview in the sidebar and, in the menu bar, next to the app's icon, and each server's card in the all-servers overview and the menu bar panel shows its own bar. When the operation ends you get a notification saying whether it finished, was cancelled, or ended with an error.

- **Array problems:** appears only when Unraid reports a disk as disabled or invalid, shown in red. Such a disk is being emulated from parity, so check the array in the Unraid web interface.
- **Temperature alerts:** appears only when a disk is at or above your warning temperature.
- **System:** hostname, Unraid version, CPU model, core and thread counts, and boot time.
- **CPU and Memory:** current load with a graph of the last 60 refreshes. With SSH on, the CPU card also shows the CPU temperature.
- **Array:** state, used and free space, and the hottest disk.
- **Docker:** running containers out of the total.
- **VMs:** running VMs out of the total.
- **Alerts:** unread notification counts split into alert, warning, and info.

System & Power

- **Hardware:** system and motherboard make and model, CPU socket and speed, architecture.
- **Software:** Unraid, kernel, and API versions.
- **License & server:** license type and expiry, flash drive vendor and product, time zone, array disk count, share count, and safe mode.
- **Services:** whether each Unraid service is online.
- **UPS:** status, battery charge, estimated runtime, load, and voltages. If no UPS is attached, the card says "No UPS detected".

Network & Temps (SSH)

- **CPU temperature:** a large reading, a history graph, and each core or package sensor. The reading turns orange above 70 C and red above 80 C.
- **Other sensors:** NVMe and any other temperature sensors the server reports.
- **Network traffic:** combined download and upload rates, a graph, and a per-interface breakdown. Virtual interfaces such as `lo`, `veth*`, `docker*`, and `virbr*` are hidden. Rates need two samples, so they appear after the second refresh.

Performance (SSH)

- **Load and uptime:** 1, 5, and 15 minute load averages and uptime.
- **CPU cores:** a usage bar for every core, grouped by the type of core and labelled with it. On a mixed ("hybrid") Intel CPU such as a Core Ultra, the bars fall under **Performance cores**, **Efficiency cores**, and **Low-power efficiency cores**, each with its core and thread count, and labels such as "P-core 2, thread 1" or "E-core 5". A performance core with hyper-threading shows two bars, one per thread. On an ordinary CPU every core is simply "Core 3", and bars are labelled "CPU 3" if the server cannot describe its layout. The layout is read once and remembered.
- **NPU:** shown only when the server has a neural processing unit with a working driver, for example the "AI Boost" NPU in Intel Core Ultra chips. It shows how busy the NPU is (a bar, the percentage, and a short history), its clock speed, the memory it is using, and its driver. If the NPU is present but

no driver is loaded, the card says so.

- **GPU:** shown for each graphics device the server reports. It shows the GPU's activity as a bar and percentage with a short history, its clock speed, memory where the driver reports it, and its name. On Intel it also shows how many cores it has, for example "7 Xe-cores · 112 execution units", when the server's debug information allows it. Linux does not report load for individual GPU cores, so the figures are for the whole GPU. For Intel GPUs, activity is the share of time the GPU was awake, which follows real work closely but slightly overstates it. AMD and NVIDIA load is reported directly (NVIDIA needs `nvidia-smi` on the server).
- **Memory:** usage with a breakdown of in-use, cache and buffers, available, shared, and dirty memory, plus swap if configured.
- **Disk I/O:** read and write speed for every physical disk, labeled with the Unraid disk name.
- **Fans:** fan speeds reported by the motherboard.
- **Top processes:** the busiest processes by CPU.

SMART Health (SSH)

One card per disk showing:

- A health badge: **Healthy**, **Warning**, **Failing**, **Standby**, or **Unknown**
- Temperature, model, serial, firmware, capacity, drive type, power-on time, power cycles, last self-test result, and error log entries
- Any concerns found (see below)
- A full table of SMART attributes (NVMe drives show wear, spare capacity, media errors, and unsafe shutdowns instead)

Buttons per disk: **Short self-test**, **Extended self-test** (asks first, can take hours), **Spin down**, and **Spin up**.

How health is decided:

- **Failing:** the drive reports an overall SMART failure, or any attribute has failed.
- **Warning:** reallocated, pending, offline-uncorrectable, or reallocation-event sector counts above zero; NVMe media errors; critical warning flags; SSD wear of 90 percent or more; or spare capacity below its threshold.
- **Healthy:** the drive passes with no concerns.

Disks that are spun down are never woken for a SMART check. Their last known data is kept and they show a "Spun down" badge. The tab checks automatically every 30 minutes, or when you click **Check now**.

Controls (SSH and Admin, depending on the action)

- **Array:** shows the state with **Start array** and **Stop array**. During any array operation it shows progress with **Pause**, **Resume**, and **Cancel** (which asks first, because progress is lost). When idle you can start a read-only check or a correcting check.

- **Disks and mover (SSH):** spin all disks up or down, and start or stop the mover.
- **Power (SSH): Reboot and Shut down.** Both ask for confirmation and use Unraid's clean powerdown, which stops the array first. A shut-down server cannot be turned back on from the app.
- **Cache clean-up (SSH):** checks your cache pools for two kinds of leftovers and lets you remove them. Press **Clean up the cache...**, pick the pools, then **Scan the cache**. The scan only reads. Results appear in two sections, each with checkboxes and its own delete button.
 - **Files left behind by the mover.** These are files on a cache pool that also exist, at the same path, on an array disk. They are what an interrupted or blocked mover leaves behind. Each is listed with the array disk that holds the copy, its size, and how its share uses the cache. Files in shares that use the mover are ticked for you. Files in other shares are listed but not ticked.
 - **Empty folders.** Folder trees with no files, and no links, anywhere inside them. After you delete files above, scan again: the folders they leave empty show up here.
 - **Never touch these top-level folders** (default `appdata,system,domains`) are not even entered during the scan, which also makes it much faster.
 - **Leave files changed in the last N minutes alone** (default 60) protects files that are still being written.
 - **Decide a file is the same as its array copy by** is either **Size and date** (quick, and what the mover preserves) or **Every byte** (slow and thorough, it reads both copies). Either way, each file is compared byte for byte with its array copy again, just before it is deleted.
 - **Only trees unchanged for N days or more** (default 7) and **Also remove empty top-level folders** (off by default) apply to empty folders.
 - A file whose array copy differs is listed under "differ from the array copy" and is never offered for deletion.
 - Deleting files is switched off while the mover is running.
- **Console (SSH):** run any command on the server and see its output. Recent commands are kept as shortcuts.

Array & Disks

- **Array card:** state and total capacity, then each parity, data, and cache disk with its status, temperature, usage bar, and details (filesystem, spinning or spun down, HDD or SSD, read and write counts, and error count).
- **Mounted filesystems (SSH):** the flash drive, Docker image, log, and `/mnt` mounts with usage.
- **Parity check history:** the last 10 checks with date, duration, speed, and errors.

Disk temperatures use the warning and critical thresholds from Settings.

Docker

Every container with its image, state, ports, auto-start badge, and created date.

- **Play and stop buttons:** start a stopped container, or stop a running one (asks first). Admin.

- **Menu (the three dots):**
 - **Restart:** uses SSH if enabled, otherwise stops and starts through the API.
 - **Pause and Resume.**
 - **View logs (SSH):** the last 300 lines, with a refresh button.

VMs

Every VM with its state. The three-dot menu on each VM offers:

- **Details (SSH):** CPU and memory summary, disks, network interfaces, VNC display, the last 60 lines of the VM log, and a **Start with array** switch.
- **Edit (SSH):** three tabs.
 - **Basics:** change vCPUs and memory.
 - **Media:** see each CD or DVD drive, eject a disc, or insert an ISO from `/mnt/user/isos`.
 - **XML:** edit the full libvirt definition. libvirt validates every save, and a bad edit is rejected.
 - **Remove VM:** deletes the definition only. Disk images are kept, and the VM must be shut off.
- **Clone (SSH):** copy a shut-off VM and its disks under a new name. See below.
- **Eject install media:** one click to eject every disc.
- **Start, Shut down, Pause, Resume, Reboot, Force stop:** power actions. Reboot and force stop ask first. Admin.
- **Open console (Screen Sharing)** and **Open console in browser:** for running VMs.

Creating a VM

Click **New VM** above the list. Choose:

- Name, operating system (Linux or Windows 10 / Server), and firmware (UEFI or legacy BIOS)
- vCPUs and memory
- A new disk (folder and size) or an existing disk image
- Disk bus: VirtIO (fastest, but Windows needs drivers) or SATA (works everywhere)
- An install ISO, and for Windows a VirtIO drivers ISO
- Network bridge and adapter type
- Whether to start with the array and whether to start the VM now

Choosing Windows switches to a SATA disk and an Intel network adapter so the installer works without extra drivers. The VM is created as a standard KVM VM with VNC graphics. GPU and USB passthrough and a TPM (needed for Windows 11) can be added afterwards in the XML editor.

If defining the VM fails, any disk image the app just created is removed. An untouched empty image left by an earlier failed attempt is replaced automatically.

Cloning a VM

The source VM must be shut off. The clone gets:

- A new name and a new unique ID

- Copies of every file-backed disk, made sparse so unused space is not copied
- A new network address (MAC addresses are removed so libvirt generates new ones)
- Its own copy of the UEFI variable store

VMs that use a physical disk device cannot be cloned automatically. Passthrough devices are copied as-is, and two VMs cannot use the same device at once. Large disks can take a long time, so keep the window open while it runs.

Opening the console

- **Screen Sharing** opens the VM in the built-in macOS Screen Sharing app using the VM's VNC port. This is the most reliable option.
- **Browser** opens Unraid's built-in web VNC page for the VM.

The VM must be running.

Deluge

A **Deluge** tab appears in the sidebar when the server runs Deluge, found by looking for a Docker container whose image or name contains "deluge" (for example the linuxserver.io image). If Deluge runs some other way, enter its address in Settings, Servers and the tab appears anyway.

It talks to Deluge's own Web UI, so the Web UI must be enabled and reachable from your Mac. It needs the Web UI password. The default is `deLuGe`, which the app uses when no password is saved. Set yours under **Deluge Web UI password** in Settings, Servers.

The panel shows:

- **Download and Upload:** current speeds with a short history graph and any speed limit.
- **Torrents:** the total, how many are downloading, seeding, paused, or in error, plus connections, DHT nodes, and free disk space.
- **The torrent list:** each torrent with its state, a progress bar, size, speeds, ETA, ratio, seeds and peers, and tracker. Errors show Deluge's message. Filter by state, sort by date added, name, progress, speed, ratio, or size, and search by name.

Actions:

- **Pause all** and **Resume all.**
- **Add magnet** adds a magnet link with Deluge's default settings.
- **The three-dot menu on a torrent:** pause or resume it, force a recheck, copy its name, or remove it. Removing asks whether to keep the downloaded files or delete them too.

The panel refreshes every few seconds, but only while it is open, so it adds no load on the server the rest of the time.

Shares

A list of shares with used and free space. Click a share to expand it.

- **Settings (SSH):** cache use, allocation method, split level, minimum free space, included and excluded disks, and SMB export and security.
- **Show spread across drives:** opens a view of how that share is spread over your disks and pools. It measures one disk at a time so you can watch progress and stop it, then shows a stacked bar and a table with each disk's size and its percentage of the share, a summary such as "1.2 TB across 4 array disks and 1 pool", and the disks the share is not on. **Scan again** repeats it and **Copy** puts the table on the clipboard. Measuring reads the folder listing on every disk that holds the share, so it can wake spun-down disks and takes a while for a large share the first time (the server remembers listings, so a repeat is much faster). **Stop** ends the measurement at once. The measurement already running on the server for the current disk finishes by itself at idle priority, and is limited to 10 minutes.
- **New share:** a form for name, comment, cache use, allocation, split level, minimum free space, disk include and exclude lists, and SMB export and security.
- **Edit:** the same form for an existing share. Settings that the form does not cover are preserved.
- **Delete:** only for empty shares. The app checks every disk first and refuses if any files remain.

Unraid reports the whole pool's used and free space for every share on it, so many shares show identical figures. When three or more shares report the same numbers, the list leaves out the per-share bar and says so, because those figures are not the share's own size. Expand a share and use **Show spread across drives** to see how much it really holds, and where.

Share changes are submitted to Unraid's own web interface endpoint from the server itself, the same way the web interface saves them. If the server does not accept a change, a banner says so.

Notifications

Unread Unraid notifications with their importance, message, and time. Archive one with the archive button, or use **Archive all**. Admin.

Logs (SSH)

The last 250 lines of the system log, with errors in red and warnings in orange. The text can be selected and copied.

The log also records every SSH login, including the app's own connections. **Hide SSH login lines** (on by default) removes those so the useful messages are visible. Turn it off to see everything.

Menu bar

With **Launch at login** turned on in Settings, the app starts with your Mac and can stay in the menu bar only, so alerts keep working without a window. The menu bar panel lists every server with a status dot, array state, CPU and memory, Docker count, and unread notifications. Click a server to open it in the dashboard. You can also open the dashboard, open Settings, open the About window, or quit. The icon changes to a crossed-out drive when any server cannot be reached.

Banners

Results of actions appear in a banner at the top of the window. Green means success, red means a failure with the server's message. Use the copy button on the banner to copy the text.

About window

Choose **About Unraid Watcher** from the app menu, the menu bar panel, or the sidebar footer. It shows the version and the copyright notice.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Settings

Open Settings with Cmd+, or from the menu bar panel. It has two tabs: **Servers** for each server you monitor, and **General** for preferences that apply to all of them.

Servers tab

The list on the left holds your servers. Use + to add one and - to remove the selected one. Click a server to edit it, then click **Save & Connect** to apply the changes and reconnect. A small status next to the button shows whether the server is connected.

Each server has its own settings and its own saved secrets:

Setting	Default	Notes
Name	Server N	Shown in the sidebar picker, the menu bar, and in notifications.
Server URL	empty	The address you use for the Unraid web interface, for example <code>http://192.168.1.10</code> . If you leave off <code>http://</code> , the app adds it.
API key	empty	Stored in the macOS Keychain. Viewer role to monitor, Admin role to use actions.
Allow self-signed certificate	on	Lets the app connect to an HTTPS server whose certificate is not trusted by macOS. This only applies to this server's address.
Read temperatures & network over SSH	off	Master switch for every SSH-based feature on this server.
SSH user	root	The account used to log in.
SSH password	empty	Optional. Stored in the Keychain. Leave it empty to use your Mac's SSH keys.
Deluge address	empty	Optional. Leave it empty and the app finds Deluge itself: it looks for a Deluge container and uses the Web UI port Docker publishes for it (8112 if none). Fill it in if Deluge runs elsewhere, uses HTTPS, or sits behind a reverse proxy, for example <code>https://deluge.home.lan</code> .
Deluge Web UI password	empty	Stored in the Keychain. When empty, Deluge's default password <code>deluge</code> is used.

Removing a server deletes its saved API key, SSH password, and Deluge password from your Keychain. Nothing changes on the server itself.

The app allows plain HTTP, so a normal home setup works without certificates. On HTTP, the API key travels unencrypted across your network. Use a trusted local network, or set up HTTPS on your server and enter an `https://` address.

How SSH login works:

- With no password set, the app runs `ssh` in batch mode and relies on your Mac's keys.
- With a password set, the app still offers your keys first, then falls back to the password. The password is passed to `ssh` through a small temporary helper script that reads it from the environment. It is never written to disk and never placed on a command line.
- The first connection to a server accepts and remembers its host key automatically.

General tab

These apply to every server.

Setting	Default	Range	Notes
Refresh every N s	10	2 to 300	How often each server is refreshed. SSH features add a little work per refresh, so very short intervals may feel heavy on a small server.
Launch at login	off		Starts Unraid Watcher when you log in to your Mac. See below.
Start in the menu bar only when launched at login	on		Appears once launch at login is on.
Notify about alerts	on		Turns all macOS notifications on or off.
Disk warning at	45 C	30 to 70	A disk at or above this is shown as hot and triggers a warning notification.
Disk critical at	55 C	35 to 80	A disk at or above this triggers a critical notification.
CPU warning at	80 C	50 to 100	Needs SSH, because CPU temperature comes from the server's sensors.
Send test notification			Sends a sample notification and asks macOS for permission if needed.

See [Alerts and notifications](#) for exactly when notifications fire.

Launch at login

- The app registers itself with macOS as a login item. You can also see and change this in **System Settings, General, Login Items**, and the two always agree.

- At login the app starts quietly in the menu bar instead of opening its window. Choose **Open Dashboard** from the menu bar icon whenever you want the window. Turn off **Start in the menu bar only when launched at login** if you would rather see the window.
- If macOS asks for approval, Settings shows a notice with an **Open Login Items** button. Switch Unraid Watcher on there.
- Login items work best when the app is in your **Applications** folder. Settings shows a tip if it is somewhere else, such as a mounted installer or the Downloads folder.
- Opening the app yourself, at any time, always shows the window as normal. Only a launch by macOS at login starts hidden.

Where data is stored

Data	Location
API key	macOS Keychain, one item per server, named <code>apiKey.<server id></code>
SSH password	macOS Keychain, one item per server, named <code>sshPassword.<server id></code>
Deluge Web UI password	macOS Keychain, one item per server, named <code>delugePassword.<server id></code>
Server list (name, URL, SSH user, certificate and SSH switches)	App preferences (<code>com.raymondmunro.unraidwatcher</code>), under <code>servers</code>
Which server is selected, thresholds, refresh interval, toggles	App preferences (<code>com.raymondmunro.unraidwatcher</code>)
SSH password helper	A temporary script in the system temp folder, containing no secrets

Nothing is sent anywhere except to your own server.

Upgrading from a single-server version

Earlier versions kept one server in plain settings. On first launch of a multi-server version, the app turns it into your first server. The name is taken from the server address, so rename it if you like. The API key and SSH password are copied into that server's own Keychain items, and the old items are removed once the copy is confirmed. macOS may ask once for your login keychain password so the app can read the old items. Choose **Always Allow**.

If you later go back to an older version, it will not find the old settings, because they were moved. Re-enter them there.

The app was also developed under earlier names. On first launch it copies any preferences it finds from those versions too.

Resetting everything

To clear all preferences:

```
defaults delete com.raymondmunro.unraidwatcher
```

To remove saved secrets, open **Keychain Access**, search for `apiKey` and `sshPassword`, and delete those items. Removing a server in Settings deletes its items for you.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Alerts and notifications

Unraid Watcher sends standard macOS notifications. Turn them on or off, and set temperature limits, on the General tab of [Settings](#). The limits apply to every server.

Every server is checked in the background all the time. When you have more than one server, each notification starts with the server's name, for example "Prime: Disk sdb running hot", so you know which one it is about.

What triggers a notification

Disk temperature

For every disk (parity, data, and cache) the app compares the temperature with your limits:

- At or above the **warning** limit (default 45 C): a "running hot" notification.
- At or above the **critical** limit (default 55 C): a "critical" notification.

To avoid repeated alerts, a disk must cool to **3 degrees below** the limit before it can alert at that level again. A disk hovering around the limit therefore notifies once, not every refresh.

The Overview tab also shows a Temperature alerts card for any disk at or above the warning limit, and disk temperatures are colored orange or red on the Array & Disks tab.

Array operations finishing

When a rebuild, parity build, parity check, or disk clear stops being reported, you get one notification: finished, cancelled, or ended with an error. Finishing is judged from the exit code Unraid records for the last operation, which needs SSH. Without SSH the notification still arrives but cannot tell success from failure, so it says only that the operation finished.

Disabled or invalid disks

If Unraid reports a disk as disabled or invalid, you get one notification naming the disk, and an **Array problems** card appears on the Overview tab. This is the most serious disk alert, because the disk's data is being emulated from parity. You are not notified again for the same disk until it has returned to normal.

CPU temperature (needs SSH)

When the hottest CPU sensor reaches the CPU limit (default 80 C), you get one notification. It re-arms after the temperature drops 3 degrees below the limit.

SMART health (needs SSH)

When a disk's SMART health changes to **Warning** or **Failing**, you get a notification naming the disk and the first concern found. The same state does not notify twice. SMART is checked every 30 minutes, and whenever you click **Check now** on the SMART Health tab.

Unraid notifications

New unread notifications with **alert** or **warning** importance are forwarded. Informational notifications are not. On the first refresh after launch, existing notifications are recorded silently so you are not flooded with old messages.

Requirements and limits

- macOS asks for notification permission the first time the app launches. If you declined, enable it in **System Settings, Notifications, Unraid Watcher**.
- Notifications are sent only while the app is running. Turn on **Launch at login** in [Settings](#) to keep monitoring after every restart. The app normally keeps running in the menu bar when its window is closed. Choose **Quit** from the menu bar panel to stop it.
- If a server is unreachable, the app cannot see problems on it. When any server is disconnected, the menu bar icon is crossed out, and that server shows as Offline in the all-servers overview.

Testing

Open Settings and click **Send test notification**. If nothing appears, check System Settings, Notifications, and make sure Focus modes are not hiding them.

Tuning tips

- Spinning hard drives are usually comfortable up to the mid 40s C. Many people use 45 C for warning and 55 C for critical, which are the defaults.
- SSDs and NVMe drives run hotter by design. If you get frequent warnings for a cache drive, raise the warning limit slightly or improve airflow.
- Your drives' own datasheets give their rated maximum temperature. Keep the critical limit safely below it.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Controls and safety

Unraid Watcher can change things on your server, not just read them. This guide explains what each action does, what it needs, and what protects you from mistakes.

Several servers

Actions always apply to the server you are viewing, which is named in the sidebar picker and at the top of the window. The confirmations for stopping the array, a correcting parity check, spinning down all disks, rebooting, and shutting down repeat the server's name. Read it before you confirm, especially if your servers have similar roles.

Permissions

Method	Needed for
Viewer API key	All read-only dashboards
Admin API key	Container start, stop, pause, and resume; VM power actions; array start and stop; parity checks; archiving notifications
SSH	Disk spin up and down, mover, reboot and shutdown, container restart and logs, SMART tests, VM details, creation, editing, cloning, media, and console, share settings and management, the command console

Each server has its own key and its own SSH setting, so you can give a server you only watch a Viewer key and no SSH, and keep Admin access for the ones you manage. If you only want to monitor, use a Viewer key and leave SSH off. Nothing in the app can then change your server.

Actions that ask for confirmation

Action	What the confirmation warns about
Stop a container	The container will stop
Stop the array	Shares, containers, and VMs become unavailable
Start a correcting parity check	Parity is rewritten to match the data disks
Spin down all disks	Disks in use spin up again on next access
Reboot the server	All services go down briefly
Shut down the server	The server powers off and cannot be turned on from the app
Reboot a VM	The VM restarts

Action	What the confirmation warns about
Force stop a VM	Equivalent to pulling the plug
Extended SMART self-test	Can take hours and slows the disk
Delete a share	Removes the share definition
Remove a VM	Removes the definition only, disks are kept
Delete cache files that are copies of array files	Shows the list first, then confirms. Each file is re-checked byte for byte against its array copy just before it is deleted
Delete empty cache folders	Shows the list first, then confirms. Only folders with nothing inside are removed
Remove a torrent in Deluge	Choose to keep the files, or delete them too (cannot be undone)

Actions that are easy to undo, such as starting a container, pausing a VM, or a short SMART test, run immediately.

Built-in safeguards

Cache clean-up: files

- A cache file is only ever deleted when an identical copy exists on an array disk. "On the array" means a real array disk (`/mnt/disk1` , `/mnt/disk2` , and so on), never another pool.
- The scan can decide by size and date, or by comparing every byte. Whichever you choose, the delete step compares the two files byte for byte again, right before deleting each one. A file is kept, and reported, if it no longer exists, has changed since the scan, has no copy on an array disk, or differs from that copy.
- Files whose array copy differs are listed for your information and are never offered for deletion.
- Files changed in the last hour (adjustable), symbolic links, files in the skipped top-level folders, and files directly in the pool root are never examined.
- Only regular files are removed, never folders, and the array copy is never touched.
- Deleting is refused while the mover is running, and the scan says so.
- Files in shares that use the mover are ticked by default. Files in shares that prefer the cache, use it exclusively, or do not use it are listed but not ticked, because for those shares the cache copy is meant to be there.

Cache clean-up: folders

- The scan only reads. Nothing is deleted until you tick folders and confirm.
- Only folders with no files and no links anywhere inside are ever listed, and each is checked again at the moment it is deleted: the delete step removes a folder only if it is empty right then, deepest first. A folder that gained a file since the scan is left alone, along with its parents.
- The pool itself, and anything outside it, can never be named: paths must sit inside the pool and may not contain `...`.

- `appdata` , `system` and `domains` are skipped by default and never entered. Recently changed folders are skipped by default.
- Empty top-level folders (the folder for a share) are left alone unless you switch that on.

Everything else

- **Shares:** delete works only on empty shares. The app checks every disk and refuses if any file remains.
- **VMs:** removing a VM never deletes its disk images. Cloning requires the source VM to be shut off. Editing a VM definition is validated by libvirt, and a bad definition is rejected without changing the VM.
- **Failed VM creation:** if defining a VM fails, any new disk image the app created is removed. An existing disk image with data in it is never overwritten.
- **Spun-down disks:** SMART checks use a standby-aware mode so sleeping disks are not woken.
- **Input checks:** share names, VM names, disk device names, and file paths are validated, and everything passed to a shell command is quoted. Names with unusual characters are refused rather than risked.
- **Power commands** use Unraid's own powerdown, which stops the array cleanly before turning off or rebooting.

The command console

The Console on the Controls tab runs exactly what you type, as the SSH user, with no filtering and no confirmation. It is as powerful as a terminal on the server. Treat it that way.

Reversibility

Action	Easy to undo?
Start, stop, pause, or restart a container or VM	Yes
Spin disks up or down	Yes
Stop the array	Yes, start it again
Parity check (read-only)	Yes, cancel it
Parity check (correcting)	No, parity is rewritten
Force stop a VM	Risky, the guest may be left inconsistent
Delete an empty share	Re-create it
Remove a VM definition	Re-create it against the same disk
Delete a cache file that is identical on the array	Yes, in the sense that nothing is lost: the identical copy stays on the array

Action	Easy to undo?
Delete an empty cache folder	Yes, it is empty, so the system recreates it when something needs it
Remove a torrent (keep files)	Re-add the torrent or magnet link
Remove a torrent and delete files	No, the files are gone
Shut down the server	Needs physical or out-of-band power on
Command console	Depends entirely on the command

Good habits

- Try new features on a test VM or an empty test share first.
- Use shut down rather than force stop for VMs whenever possible.
- Keep an up-to-date backup of your flash drive. VM and share definitions live there.
- After editing a VM's XML, check the VM in the Unraid web interface the first time.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Troubleshooting

Every page shows problems as orange messages at the top. Problems with optional extras (license, services, UPS, parity history, and per-disk or per-container details) are grouped into one grey, collapsed line so they do not clutter the page. Click **Copy all errors** to copy every message at once.

Several servers

- **One server shows Offline in the all-servers overview:** only that server has a problem. Open it and read the orange messages, or check its address and key in Settings, Servers.
- **A server I expect is missing after upgrading:** servers from a single-server version become one server named after its address. If the list is empty, add it again with + in Settings, Servers.
- **Notifications do not say which server:** the name is added only when you have two or more servers.
- **I changed a server's settings and nothing happened:** click **Save & Connect**. Edits are applied only when you click it, and switching to another server in the list discards unsaved edits.

Connection problems

"A server with the specified hostname could not be found"

The **Server URL** for that server in Settings, Servers is wrong, or your Mac cannot resolve the name. Use the same address you type in a browser to reach Unraid, for example `http://192.168.1.10`. Try the IP address instead of a name if `tower.local` does not resolve.

"Unauthorized (HTTP 401)" or "(HTTP 403)"

The API key is wrong, expired, or lacks permission. Create a new key in Unraid under **Settings, Management Access, API Keys**, paste it into Settings, and click **Save & Connect**.

Timeouts or "could not connect"

- Check that the server is on and reachable: open its web interface from your Mac.
- Confirm the address, and the port if you changed it (for example `http://192.168.1.10:8080`).
- If you use HTTPS with a self-signed certificate, make sure **Allow self-signed certificate** is on.

The window just says "Connect to your server"

No URL or API key is saved yet. Open Settings and fill in both.

Unexpected messages that are not real problems

Message	Meaning
"No UPS data returned from apcaccess"	No UPS is attached. The app shows "No UPS detected" instead of an error.
Optional details unavailable	Your Unraid version does not offer one of the extra data fields. Everything else keeps working.

API data errors

Each panel fetches its data separately, so one failing field affects only its own panel.

"Cannot return null for non-nullable field ..."

Your server returned nothing for a field that the API says must always exist. This is a server-side quirk with that field. The app has been adjusted to avoid the fields known to do this (for example the flash drive GUID). If you see another one, copy the message and report it.

A panel says a field "doesn't exist" or the query is invalid

The Unraid API changes between versions, and a few panels use fields that older or newer versions may not offer. Update Unraid to 7.2 or later if you can. Copy the message so the query can be adjusted.

Actions fail with a red banner

Actions that go through the API (container start and stop, VM power, array, parity check, archive notification) need an **Admin** API key. A Viewer key can read but not change anything. The banner shows the server's own message, and the copy button on the banner copies it.

SSH problems

- **The server's log is full of SSH logins:** older versions logged in again on every refresh. Current versions reuse one connection. The Logs tab also hides these lines by default.
- **A refresh pauses for a moment after the server restarts or you change its SSH password:** the shared connection has to be made again. It recovers by itself.

The SSH error message is shown exactly as `ssh` reported it.

Message	Fix
Could not resolve hostname	The Server URL is wrong. SSH uses the same host.
Permission denied	Wrong user or password. Check the SSH user and password in Settings.

Message	Fix
Connection refused	SSH is off on the server. Turn on Use SSH in Unraid, Settings, Management Access.
Connection timed out	The server is unreachable or a firewall blocks port 22.
Host key verification failed	The server was rebuilt and its key changed. Remove the old entry from <code>~/.ssh/known_hosts</code> on your Mac.

To test SSH outside the app:

```
ssh root@your-server-address
```

If that works in Terminal but not in the app, check that the user and password in Settings match.

Keychain prompts

macOS asks for your login keychain password when the app reads a saved secret for the first time after it was rebuilt or renamed. Enter it and choose **Always Allow**. If you build the app yourself, signing with a stable certificate (the build script does this automatically when an Apple Development certificate is installed) stops the prompt from coming back after each rebuild. Ad-hoc signed builds look like a different app every time.

Notifications

If nothing appears:

1. Click **Send test notification** in Settings.
2. Check **System Settings, Notifications, Unraid Watcher** and allow alerts.
3. Make sure a Focus mode is not suppressing them.
4. Confirm **Notify about alerts** is on.

Deluge

- **There is no Deluge tab:** the app shows it only when it sees a Docker container with "deluge" in its image or name, or when you enter a Deluge address in Settings, Servers. Check the container's name and image, or fill in the address.
- **"Deluge rejected the Web UI password":** enter the Web UI password under **Deluge Web UI password** in Settings, Servers. The default is `deluge`. This is the Web UI password, not the daemon password.
- **"Connection refused" or a timeout:** the Web UI is not reachable at the address shown in the panel. Check that the container is running and publishes the Web UI port (8112 by default), or set the right address in Settings.

- **"Doesn't look like the Deluge Web UI":** something else answered at that address. Check the port, and that any reverse proxy forwards the `/json` path.
- **"No Deluge daemon to connect to":** the Web UI is running but is not linked to a daemon. Open the Deluge Web UI once in a browser and use its Connection Manager to add the daemon.
- **Pause, resume, or remove fails:** the banner shows Deluge's own message. Copy it with the button on the banner.

CPU core types, NPU, and GPU

- **The core bars say "CPU 0", "CPU 1" instead of core types:** the server's core layout could not be read, which usually means SSH was off or briefly unavailable the first time. Reopen the Performance tab after fixing SSH, or restart the app.
- **All the small cores are called "E-core" and none are "LP E-core":** the server did not report clock speeds for them, so the two kinds cannot be told apart. The app does not guess.
- **My CPU is not hybrid but shows groups:** it should show a single list. If it does not, tell me the CPU model so the detection can be improved.
- **There is no NPU card:** the card appears only when an NPU with a driver is found. If the NPU exists but is not usable, a card says that. Linux needs its `intel_vpu` driver for Intel NPUs, and the kernel in your Unraid version has to include it.
- **The NPU card says it does not report how busy the NPU is:** the driver is loaded but older than the one that provides a busy counter.
- **The NPU or GPU shows "measuring...":** the percentage needs two readings a few seconds apart. It appears after the next refresh.
- **There is no GPU card:** the server reported no graphics device, or it is a type this check does not know. Virtual machine graphics adapters are not listed.
- **The GPU card has no core count:** the count comes from the kernel's debug information, which is not always mounted or readable. The rest of the card still works.
- **The GPU activity looks too high on Intel:** it measures time spent awake rather than real work, so it can read a few points high. Compare it with the Intel GPU Top plugin if you need exact engine figures.
- **NVIDIA shows no load:** the load comes from `nvidia-smi`, which must be installed on the server. Without it the card shows only the device.

Array operation progress

- **There is no progress card during a rebuild:** the app reads it over SSH, or from the Unraid API when SSH is off. Older API versions may not offer it, so turn on SSH.
- **"working out the time left":** the estimate needs about 20 seconds of readings. Until then the speed Unraid reports is used.
- **The time left changes a lot:** rebuilds slow down when the array is busy, and the estimate follows the real speed. A rebuild of a large disk takes days.

- **Pause or Cancel did nothing, or showed an error:** those buttons use the Unraid API's parity controls, which are made for parity checks. If your version does not apply them to a rebuild, use the Unraid web interface.

Share spread and cache clean-up

- **Measuring a share is slow the first time and fast the second:** the server keeps recently read folder listings in memory, so a repeat measurement is nearly instant.
- **Measuring a share is slow:** it reads every file name on each disk that holds the share. Large shares on spinning disks can take minutes. Use **Stop** to cancel, and the disks measured so far stay on screen.
- **The disks woke up:** measuring has to read the disks. Do it when that does not matter.
- **The clean-up scan takes a long time:** it walks every folder on the pools. Keep your big folders (such as `appdata`) in the skip list so they are never entered. The scan runs at low priority and gives up after 25 minutes.
- **A folder I expected is not listed:** it is hidden by the age setting, the skip list, or the top-level switch, or it holds a file or a link somewhere inside. The summary says how many were hidden by your settings.
- **"No cache file has an identical copy on the array":** nothing on the cache, outside the skipped folders, is also on the array. That is the normal, healthy state after the mover has run.
- **Deleting files is switched off, and the sheet says the mover is running:** wait for the mover to finish, then scan again. Deleting while it runs could race with it.
- **Files appear under "differ from the array copy":** the same path exists on the cache and on the array with different contents. They are never offered for deletion. Compare them yourself and decide which to keep.
- **A file was kept when I deleted:** the sheet's message gives the reason: it changed since the scan, its array copy is missing or different, or it no longer exists. Scan again.
- **The comparison ran out of time:** the file check stops after 15 minutes and keeps what it found. Use the quick comparison, or add folders to the skip list.
- **This check needs the array to be started:** the array disks must be mounted to compare against them.
- **The pool I want is not offered:** pools come from the server's mounted filesystems. The pool must be mounted, which means the array is started.
- **"The scan didn't finish":** the server stopped answering part-way. Try again, with more folders in the skip list.

Launch at login

- **The switch turns itself off or an error appears:** move the app to your **Applications** folder, open it from there, and try again.
- **Settings says macOS needs your approval:** open **System Settings, General, Login Items** and switch Unraid Watcher on. The **Open Login Items** button in Settings takes you there.

- **The window opens at login even though you wanted the menu bar only:** check that **Start in the menu bar only when launched at login** is on. If the window still appears, close it, and the app keeps running in the menu bar.
- **It did not start after a restart:** check that it is still listed and switched on in Login Items. Moving or replacing the app can drop the entry. Turn the option off and on again in Settings.

Data that looks empty

Symptom	Reason
Network rates empty	Rates need two samples. Wait for one more refresh.
CPU core bars and disk I/O empty	Same, they are computed from the difference between two samples.
SMART shows "Standby"	The disk is spun down and was deliberately not woken. Its last data is kept when available.
Temperatures missing	SSH is off, or the server exposes no sensors the app recognizes.
Share settings missing	SSH is off, or the share uses defaults and has no settings file.
Most shares show no usage bar	They report the same pool-wide figures. Use Show spread across drives on a share for its real size.
Torrents show "Size not known yet"	Deluge has not downloaded their metadata yet, so it does not know their size.

VM problems

Creating a VM failed

The banner shows libvirt's own message. Common causes:

- **"A VM named ... already exists":** pick another name.
- **"A disk image already exists at ...":** a disk with data is already at that path. Choose another folder or name, or use the existing-image option.
- **A bridge or network error:** pick one of the bridges listed in the form. They come from your server.
- **Missing UEFI firmware files:** choose legacy BIOS, or check that your Unraid version includes OVMF.
- **Permission or path errors:** check the disk folder exists on a share that can hold VM disks.

Older builds of the app could fail with "per-device boot elements cannot be used together with os/boot elements". That was fixed. Update to the latest version.

If creation fails after the disk image was made, the app deletes the new image. An empty image left by an earlier failed attempt is replaced automatically.

Clone failed

- The source VM must be shut off.
- VMs that use a physical disk device cannot be cloned automatically.
- Check there is enough free space for the copied disks.

The browser console button opens a blank or broken page

Use **Open console (Screen Sharing)** instead. It connects straight to the VM's VNC port and does not depend on web interface paths.

Edited XML was rejected

libvirt refuses invalid definitions and leaves the VM unchanged. The banner shows what it disliked. Common causes are a CPU topology that no longer matches the vCPU count, or references to CPUs that were removed.

Shares

"The server didn't accept the change"

Share changes are posted to Unraid's own web interface endpoint from the server. If that fails, check the change in the Unraid web interface to confirm what state the share is in, then try again. Share names can contain letters, numbers, spaces, underscores, hyphens, and periods.

Delete is refused

The share still contains files on at least one disk. Empty it first. This protects your data.

Building problems

- **Codesign cannot find an identity:** the script falls back to ad-hoc signing automatically. Install an Apple Development certificate (free with an Apple ID in Xcode) for stable signing.
- **The DMG window is not styled:** macOS asks to let your terminal control Finder the first time. Allow it, or accept the plain DMG, which still installs normally.
- **Gatekeeper blocks the app on another Mac:** right-click the app and choose Open the first time. Removing the warning for everyone needs a Developer ID certificate and notarization.

Still stuck?

Click **Copy all errors** and, if relevant, copy the banner text of the failing action. Those two pieces of text usually identify the problem.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Data sources

This page lists everything Unraid Watcher asks of your server, so you can see exactly what it does and audit it. There are two channels: the Unraid GraphQL API and SSH.

GraphQL API

All requests go to `<server>/graphql` over HTTP or HTTPS with the API key in the `x-api-key` header.

Queries (read only)

Panel	Data requested
Overview	<code>info</code> (OS hostname, distribution, release, uptime; CPU brand, cores, threads) and <code>metrics</code> (CPU percent, memory total, used, percent)
Array	<code>array</code> (state, capacity, parities, disks, caches with name, device, size, temperature, status, filesystem usage)
Docker	<code>docker.containers</code> (id, names, state, status, image)
Container details	<code>docker.containers</code> (id, auto-start, created, ports)
VMs	<code>vms.domains</code> (id, name, state)
Shares	<code>shares</code> (name, free, used, size, comment)
Notifications	<code>notifications</code> (unread counts and the 20 newest unread items)
System details	<code>info</code> (baseboard, system, OS kernel and architecture, CPU manufacturer, speed, socket, versions)
Disk details	<code>array</code> <code>disks</code> (filesystem type, errors, spinning state, reads, writes, rotational)
UPS	<code>upsDevices</code> (name, model, status, battery, power)
Parity history	<code>parityHistory</code> (date, duration, speed, status, errors)
Parity status	<code>array.parityCheckStatus</code> (running, paused, progress, errors, status)
License	<code>registration</code> (type, state, expiration)
Services	<code>services</code> (name, online, version)
Flash	<code>flash</code> (vendor, product)
Array operation (without SSH)	<code>vars</code> (the <code>mdResync</code> values: action, position, size, speed window, correcting)
Server	<code>vars</code> (name, version, registration, time zone, disk and share counts, array filesystem state, safe mode)

Each query runs independently and concurrently on every refresh, so a failure in one does not affect the others.

Mutations (changes, need Admin)

Action	Mutation
Start, stop, pause, resume a container	<code>docker { start, stop, pause, unpause }</code>
VM power	<code>vm { start, stop, pause, resume, forceStop, reboot }</code>
Start or stop the array	<code>array { setState(desiredState: START or STOP) }</code>
Parity check	<code>parityCheck { start(correct), pause, resume, cancel }</code>
Archive a notification	<code>archiveNotification(id)</code>

SSH

Commands run as the SSH user through `sh -s`, with the script sent on standard input.

The app keeps one SSH connection open and reuses it for every command (OpenSSH connection sharing). It logs in once, rather than once per refresh, so the server's log is not filled with a login for every refresh and each refresh is faster. The connection closes by itself two minutes after its last use. Its control socket lives in a private folder, `/tmp/uw-<your user id>`, and nothing else is shared.

Polled on every refresh (when SSH is on)

A single connection reads, in one batch:

- `/proc/net/dev` for network counters
- `/sys/class/hwmon` for temperatures and fan speeds
- the array state in `/var/local/emhttp/var.ini` (the `mdResync` values that describe a rebuild, parity build, or check)
- `/proc/loadavg` and `/proc/uptime`
- `/proc/meminfo` for memory detail
- `/proc/stat` for per-core CPU
- `df` for mounted filesystems
- `/sys/class/accel` for the NPU: its busy time, clock speed, and memory in use (the `intel_vpu` driver), plus a scan of the PCI bus for processing accelerators
- `/sys/class/drm` for each GPU: clock speed and idle time (Intel `i915` and `xe`), busy percent and memory (AMD), and `nvidia-smi` for NVIDIA cards when it is installed
- `/sys/kernel/debug/dri` for an Intel GPU's core count, when the kernel's debug files are readable
- `/proc/diskstats` for disk I/O
- `ps` for the top processes
- the last 250 lines of `/var/log/syslog`

On demand

Feature	Commands
SMART check	<code>smartctl -j -a -n standby /dev/<device></code> for each disk (the standby option leaves sleeping disks alone)
SMART self-test	<code>smartctl -t short</code> or <code>smartctl -t long</code>
Spin down	<code>hdparm -y /dev/<device></code>
Spin up	a single-sector direct read with <code>dd</code>
Mover	<code>mover start</code> , <code>mover stop</code>
Power	<code>powerdown</code> or <code>powerdown -r</code> , started in the background
Container logs and restart	<code>docker logs --tail 300</code> , <code>docker restart</code>
Console	whatever you type
Share settings	reading <code>/boot/config/shares/*.cfg</code>
Share disk usage	<code>du -sk /mnt/*/<share></code>
Share create, edit, delete	a request to Unraid's own web interface endpoint on the server (<code>/update.htm</code> , over its local socket when available), authenticated with the server's own token
CPU core layout	Read once per server: <code>/sys/devices/system/cpu/cpu*</code> (thread siblings, top speed, capacity) and <code>/sys/devices/cpu_core</code> and <code>cpu_atom</code> , which list the performance and efficiency cores on a hybrid Intel CPU
Share spread	A listing of <code>/mnt/*/<share></code> to find where the share lives, then <code>du -sk</code> on each location, one at a time, at idle priority (<code>nice</code> , <code>ionice</code>) with a 10 minute <code>timeout</code> when those exist
Cache clean-up scan	<code>find</code> , <code>sort</code> , <code>awk</code> , <code>comm</code> and <code>stat</code> over the pool, read-only, at low priority (<code>nice</code> , <code>ionice</code>) and with a <code>timeout</code> when those exist
Cache clean-up: files already on the array, scan	<code>find</code> over the pool (skipping your skipped folders), then for each file a check of <code>/mnt/disk*</code> for the same path, comparing size and date (<code>stat</code>) or every byte (<code>cmp</code>). Read-only, at idle priority with a 15 minute limit
Cache clean-up: files already on the array, delete	For each file: <code>stat</code> and <code>cmp</code> against the copy on an array disk, then <code>rm -f</code> of that single file only if they match. Refuses if the mover (<code>pgrep</code>) is running
Cache clean-up delete	<code>find <folder> -depth -type d -empty -delete</code> , so only folders that are empty at that moment are removed
Share delete safety check	<code>find</code> over the share's folders on each disk
VM details	<code>virsh dominfo</code> , <code>domblklist</code> , <code>domiflist</code> , <code>vncdisplay</code> , and the libvirt VM log

Feature	Commands
VM autostart	<code>virsh autostart</code>
VM create	<code>virsh dominfo</code> (name check), <code>qemu-img create</code> , <code>virsh define</code> , optional <code>virsh autostart</code> and <code>virsh start</code>
VM edit	<code>virsh dumpxml --inactive</code> , <code>virsh define</code>
VM remove	<code>virsh domstate</code> , <code>virsh undefine --nvram</code>
VM media	<code>virsh domblklist --details</code> , <code>virsh change-media</code>
VM console	<code>virsh dumpxml</code> to read the VNC and websocket ports
VM clone	<code>virsh dominfo</code> , <code>virsh domstate</code> , <code>cp --sparse=always</code> , <code>virsh define</code>
ISO and bridge lists	<code>ls /mnt/user/isos</code> , and the bridge devices under <code>/sys/class/net</code>

Values that come from you or from the server (names, paths, device names) are validated and shell-quoted before they are used in a command.

Deluge

When a server has a Deluge tab open, the app talks straight to Deluge's Web UI over HTTP (or HTTPS, if you gave an `https` address), on the Web UI port, at the path `/json`. This does not go through the Unraid API or SSH. Requests are JSON-RPC calls, and the Web UI keeps your login in a session cookie that the app holds in memory only.

Purpose	Calls
Log in	<code>auth.login</code>
Torrent list and speeds	<code>web.update_ui</code> (one call returns every torrent plus session statistics)
Attach the Web UI to the Deluge daemon if it is not attached	<code>web.get_hosts</code> , <code>web.connect</code>
Pause and resume	<code>core.pause_torrents</code> , <code>core.resume_torrents</code> (older Deluge: <code>core.pause_torrent</code> , <code>core.resume_torrent</code>)
Pause all and resume all	<code>core.pause_session</code> , <code>core.resume_session</code> (older Deluge: <code>core.pause_all_torrents</code> , <code>core.resume_all_torrents</code>)
Recheck	<code>core.force_recheck</code>
Remove, with or without files	<code>core.remove_torrent</code>
Add a magnet link	<code>core.add_torrent_magnet</code>

The Deluge password travels to the Web UI in clear text if you use plain HTTP, like the Unraid API key. Use a trusted local network or an `https` address.

What stays on your Mac

- Fetched data is held in memory only. Nothing is written to disk except your settings and the Keychain items.
- The app contacts no server other than the ones you add.
- Each server has its own Keychain items, and nothing is shared between servers.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

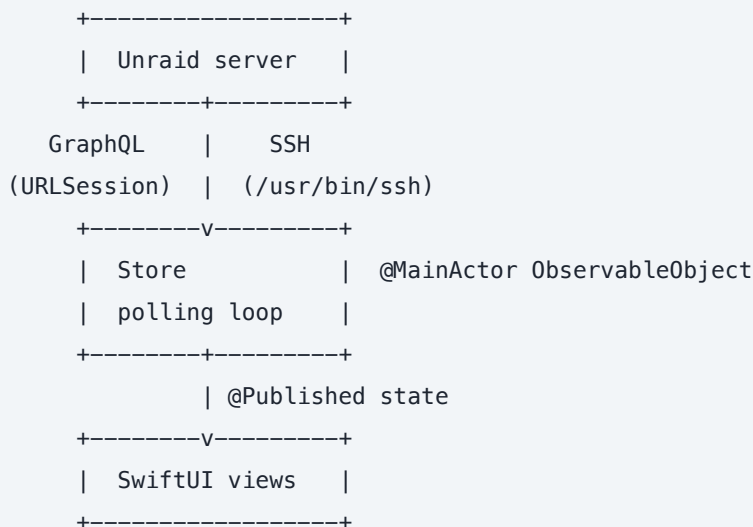
Architecture

Unraid Watcher is a single-target Swift Package that builds a SwiftUI macOS app. It has no third-party dependencies. It uses only Apple frameworks (SwiftUI, Charts, AppKit, Security, UserNotifications, Foundation).

Project layout

Package.swift	Swift package (macOS 14, one executable target)
build_app.sh	Builds and signs "Unraid Watcher.app"
make_dmg.sh	Builds the DMG installer
Resources/AppIcon.icns	App icon
tools/make_icon.swift	Draws the icon and writes the .icns
tools/make_dmg_bg.swift	Draws the DMG window background
tools/make_docs_pdf.py	Builds the documentation PDF from the markdown files
docs/assets/icon.png	App icon used in the docs and the PDF cover
docs/Unraid-Watcher-Documentation.pdf	The combined documentation
Sources/UnraidWatcher/	
App.swift	App entry, scenes, About panel, settings migration
Client.swift	GraphQL client, API models, Keychain helper
Servers.swift	Server profiles, ServerManager, server picker, overview, menu bar, Settings
Store.swift	State, polling loop, and alerts for one server
Remote.swift	SSH execution and parsing of server output
Actions.swift	Actions (API and SSH), SMART models and parser
LoginItem.swift	Launch at login and quiet start in the menu bar
Deluge.swift	Deluge Web UI client, models, and the Deluge panel
ArrayOperation.swift	Progress of a rebuild, parity build, or check, and the card that shows it
CPUTopology.swift	Which kind of core each CPU is, and the core bars
NPU.swift	Reading and showing a neural processing unit
GPU.swift	Reading and showing graphics processors
Spread.swift	Share spread: per-disk measuring and the spread view
CacheCleanup.swift	Empty-folder scan and delete scripts, and the clean-up sheet
StrayFiles.swift	Finding cache files that are already on the array, and the verified delete
ShareVM.swift	Share management and VM detail views
VMEditor.swift	VM creation, XML generation, edit sheet
VMExtras.swift	VM clone planning, media, console
Views.swift	Sidebar, all tab views, menu bar, settings

Data flow



Servers

`ServerManager` owns the list of `ServerProfile` values (name, URL, SSH settings), saved as JSON in preferences, with each server's API key and SSH password in its own Keychain items. It creates one `Store` per server and starts every one, so all servers poll and alert in the background. It also tracks which server is selected, rebuilds a server's store when its settings are saved, and migrates the old single-server settings on first launch.

The window shows the selected server's `Store` through the SwiftUI environment, so the existing views did not need to know about multiple servers. `AllServersView` and the menu bar read every store directly.

The Store

Each `Store` is the source of truth for one server. A `Task` loop calls `refresh()` every N seconds.

- **API data:** each GraphQL query runs as its own concurrent task (`async let`). Each returns a `Result`, so a failing query records an entry in the `errors` dictionary without affecting the others. Views show errors relevant to the current tab; errors for optional extras are grouped into a collapsed notice.
- **SSH data:** if SSH is enabled, one `ssh` call fetches many sections at once. The output is parsed into a `RemoteSample`. Rates (network, disk I/O, per-core CPU) are computed from the difference between the current and previous samples and the elapsed time.
- **History:** short arrays (the last 60 samples) feed the sparkline charts.
- **Alerts:** after each refresh, `checkAlerts()` compares temperatures and notifications against thresholds and sends macOS notifications. A small state machine with hysteresis prevents repeats.
- **SMART:** loaded separately every 30 minutes when SSH is on, or on demand.

Talking to the server

- `UnraidClient` (in `Client.swift`) posts GraphQL documents with an `x-api-key` header. It tolerates self-signed certificates only when the setting is on. Responses decode through `Envelope<T>`, and numeric fields that Unraid sometimes returns as strings decode through the `Flex` and `LStr` types.
- `Remote.exec` (in `Remote.swift`) runs `/usr/bin/ssh` as a subprocess and sends the script on standard input to `sh -s`. Standard error is drained on a separate thread to avoid pipe deadlocks. A cancelled task stops its ssh process and returns to the caller at once, even though a shared connection keeps the command's output open, which is how the Stop buttons work. The command already running on the server finishes by itself, so long scans run at idle priority with a time limit. It shares one authenticated connection between commands (`ControlMaster=auto`, kept alive for two minutes, with the socket in a private `/tmp/uw-<uid>` folder), so there is one login rather than one per refresh. In key mode it uses batch mode. In password mode it uses a temporary `SSH_ASKPASS` helper that reads the password from an environment variable. `ServerAliveInterval` keeps long operations such as VM cloning connected.
- Anything interpolated into a shell script goes through `shq()` (single-quote escaping), and names, paths, and device names are validated first (`validShareName`, `validPath`, `validDev`).

Deluge

`DelugeClient` speaks JSON-RPC to the Deluge Web UI with its own `URLSession`, which keeps the login cookie. It logs in again automatically when the session expires, attaches the Web UI to the daemon if needed, and falls back to older method names for pause and resume. The `Store` creates a client per server (rebuilt when the address or password changes) and detects Deluge from the Docker container list. The panel polls only while it is on screen.

Parsers and generators

These are written as pure functions so they are easy to test in isolation:

Function	Purpose
<code>Remote.parse</code>	Turns the batched SSH output into a <code>RemoteSample</code>
<code>Smart.parse</code>	Turns <code>smartctl -j</code> JSON into a <code>SmartInfo</code> with a health verdict
<code>Store.parseCfg</code>	Parses Unraid share <code>.cfg</code> files
<code>domainXML</code>	Generates a libvirt domain definition for a new VM
<code>createScript</code>	Builds the shell script that creates a VM and cleans up on failure
<code>planClone</code>	Computes the XML and file copies needed to clone a VM
<code>cleanupScanScript</code> , <code>parseEmptyRoots</code>	Build the read-only empty-folder scan and parse its output

Function	Purpose
<code>cleanupDeleteScript</code> , <code>validCleanupPath</code>	Build the delete step and check every path stays inside the pool
<code>classifyCores</code> , <code>CPUTopology.parse</code>	Decide from the layout files which logical CPUs are performance, efficiency, or low-power cores, and number them
<code>NPUProbe.parse</code> , <code>npuBusyPercent</code>	Read NPU details, and turn its busy-time counter into a percentage
<code>GPUProbe.parse</code> , <code>gpuActivePercent</code>	Read GPU details for Intel, AMD, and NVIDIA, and turn Intel's idle-time counter into an activity percentage
<code>ArrayOperation.from</code>	Turns the array state values into an operation with a title, progress, and what it means for your data
<code>strayScanScript</code> , <code>parseStrayScan</code>	Find cache files that also exist on an array disk, and whether they match
<code>strayDeleteScript</code>	Re-verify each file against its array copy, byte for byte, and remove only those that still match
<code>spreadLocationsScript</code> , <code>parseSpreadLocations</code>	Find where a share lives, skipping merged views and unassigned devices
<code>DelugeParse.snapshot</code>	Turns a Deluge <code>web.update_ui</code> result into torrents and statistics

The project does not yet include an automated test target. During development these functions were checked with throwaway harnesses (sample inputs, `xmlLint` , and stand-in `virsh` and `qemu-img` scripts). Turning them into XCTest cases is a good next step.

UI structure

- `ContentView` is a `NavigationSplitView` . The sidebar lists sections, and the detail area switches on the selected `Section` .
- A banner at the top of the detail area shows action results.
- `ErrorsPanel` decides which errors are shown on which tab.
- Reusable pieces: `Card` , `UsageBar` , `Pill` , `Spark` .
- Sheets handle multi-field flows: share editor, VM create, VM edit, VM clone, VM details, container logs.
- `MenuBarExtra` provides the menu bar item. `Settings` provides the preferences window.

Security design

- Secrets (API key, SSH password) are stored in the Keychain, separately for each server.
- The SSH password reaches `ssh` only through an environment variable read by a helper script, so it is not written to disk or shown in process arguments.
- Destructive operations require explicit confirmation in the UI.

- The app connects only to the address you enter.

Adding a feature

1. Add the data model and a query or script. Prefer a separate, isolated query so a failure does not break other panels.
2. Add `@Published` state to `Store` and fill it in `refresh()` (or load it on demand from an action).
3. If it can fail, give it an error key, and decide in `ErrorsPanel` and `relevant(_:)` where it should show.
4. Add the view, and a sidebar `Section` if it needs its own tab.
5. For anything destructive, add a confirmation dialog.
6. Document the new queries or commands in [Data sources](#).

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Building and releasing

Requirements

- A Mac running macOS 14 or later
- Xcode (the full app, for the Swift toolchain and SDK) or the matching command line tools
- Optional: an Apple Development certificate, for stable code signing

Build the app

```
./build_app.sh
```

The script:

1. Builds a release binary with `swift build -c release`.
2. Creates `Unraid Watcher.app` with the executable, the icon, and an `Info.plist`.
3. Signs it with the first Apple Development certificate it finds, or ad-hoc if there is none.

Open it with:

```
open "Unraid Watcher.app"
```

For a quick debug build while developing, use `swift build` and run the executable from `.build`. Features that depend on being an app bundle (notifications, the app icon) only work from the `.app`.

Why signing matters

macOS ties Keychain access to the app's code signature. An ad-hoc signature changes on every build, so macOS asks for your login keychain password again after each rebuild. A stable Apple Development signature avoids that. To see your available identities:

```
security find-identity -v -p codesigning
```

Build the DMG installer

```
./make_dmg.sh
```

This builds the app and the documentation PDF, then produces `dist/Unraid-Watcher-<version>.dmg` containing the app, an Applications shortcut, `Documentation.pdf`, `LICENSE.txt`, and a styled background with an arrow. Steps performed:

1. Build and sign the app, and build the documentation PDF.
2. Draw the background (`tools/make_dmg_bg.swift`).
3. Create a writable disk image and arrange the window using Finder scripting.
4. Compress it to a read-only image and sign it.

The first time, macOS may ask permission for your terminal to control Finder. Allow it for the styled window. If you decline, the DMG is still created and installs normally, just without the custom layout.

Build the documentation PDF

```
python3 tools/make_docs_pdf.py
```

This combines the README, every guide in `docs/` , and the changelog into `docs/Unraid-Watcher-Documentation.pdf` , with a cover page showing the app icon, a contents page, and page numbers. It needs only Python 3 and Google Chrome (used headless to print the PDF). Rebuild it whenever the documentation changes, then commit the new PDF.

`make_dmg.sh` runs this step automatically and puts the PDF in the installer as `Documentation.pdf` , next to a copy of the license as `LICENSE.txt` .

Regenerate the icon

```
swift tools/make_icon.swift
```

This redraws the icon in code and rewrites `Resources/AppIcon.icns` . Edit the drawing code in `tools/make_icon.swift` to change the design.

Changing the version

The version number appears in five places. Update all of them:

File	What to change
<code>build_app.sh</code>	<code>CFBundleShortVersionString</code> (and <code>CFBundleVersion</code> , the build number) in the generated <code>Info.plist</code>
<code>make_dmg.sh</code>	<code>VER=</code> at the top (also names the DMG file)
<code>Sources/UnraidWatcher/App.swift</code>	<code>.applicationVersion</code> in the About panel
<code>Sources/UnraidWatcher/Servers.swift</code>	The version text at the bottom of the General tab in Settings
<code>tools/make_docs_pdf.py</code>	<code>VERSION</code> , shown on the PDF cover

Also add an entry to [CHANGELOG.md](#).

Distributing to other Macs

A build signed with an Apple Development certificate runs on your own Mac. Other Macs show an unidentified developer warning, and the recipient must right-click the app and choose Open the first time.

To remove the warning for everyone you need a paid Apple Developer Program membership and these steps:

1. Sign with a **Developer ID Application** certificate and the hardened runtime:

```
codesign --force --options runtime --sign "Developer ID Application: Your Name (TEAMID)" "Unraid-Watcher.app"
```

2. Submit the DMG for notarization:

```
xcrun notarytool submit dist/Unraid-Watcher-1.4.0.dmg --keychain-profile "your-profile" --wait
```

3. Staple the ticket:

```
xcrun stapler staple dist/Unraid-Watcher-1.4.0.dmg
```

Files that are not committed

`.gitignore` excludes `.build/`, `dist/`, and `*.app`. They are regenerated by the scripts above.

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.

Changelog

All notable changes to Unraid Watcher are listed here.

1.4.0

New

- **Core types:** the CPU core bars on the Performance tab are now grouped and labelled by the type of core: performance cores, efficiency cores, and low-power efficiency cores on a hybrid Intel CPU, with each core's threads shown ("P-core 2, thread 1", "E-core 5"). Other CPUs show "Core N".
- **NPU:** a card for the server's neural processing unit (such as Intel's AI Boost) with how busy it is, its clock speed, memory in use, and driver, when the driver provides them.
- **GPU:** a card for each graphics device with its activity, clock speed, memory where reported, and name, and on Intel the number of cores (Xe-cores and execution units) when the kernel allows it. Intel, AMD, and NVIDIA are supported.

1.3.0

New

- **Array operation progress:** while the array is rebuilding a disk, building parity, running a parity check, or clearing a disk, the Overview shows a progress bar with the percentage, how much is done, the speed, and about how long is left, with a note on what it means for your data. The percentage also shows beside Overview in the sidebar, in the menu bar, and on each server's card. You get a notification when it ends. The Controls tab shows the same progress with Pause and Cancel.
- **Share spread:** open any share and see how it is spread across your drives, measured one disk at a time with a stacked bar, a table of sizes and percentages, and the disks it is not on. You can stop it part-way.
- **Cache clean-up for files the mover left behind:** the same scan now also finds files on your cache pools that already exist, at the same path, on an array disk. They are listed with the disk that holds the copy. A file is only ever offered, and only ever deleted, when the array copy matches, and the delete step compares the two byte for byte again just before removing each file. Files that differ, changed recently, sit in skipped folders, or are links are never touched, and deleting is refused while the mover is running.
- **Cache clean-up:** scan your cache pools for empty folders and remove the ones you choose. The scan only reads, skips `appdata`, `system` and `domains` by default, ignores recently changed folders, and deletes only folders that are still empty at the moment of deletion.

Fixed

Found by running the app against a real server.

- **Stop buttons:** stopping a long scan or measurement returns at once. Before, with a shared SSH connection, a cancelled command kept the app waiting until the command ended on its own.
- **Controls:** the confirmations for stop array, a correcting parity check, spin down all disks, reboot, and shut down never appeared, so those buttons did nothing. They now ask first, and the question names the server. Nothing ran in the meantime, so no action was taken by accident.
- **SSH:** one connection is now shared and reused, instead of a new login on every refresh. This stops the server's log filling with login lines and makes refreshes faster.
- **Logs:** shows the last 250 lines and hides the app's own SSH login lines by default, with a switch to show them.
- **Array problems:** a disk Unraid reports as disabled or invalid now shows in red, in an Array problems card on the Overview, and triggers a notification. It was grey before.
- **Array:** the parity disk's size was shown in the wrong unit (GB instead of TB).
- **Shares:** when many shares report the same pool-wide figures, the misleading per-share bars are hidden and a note explains why.
- **Deluge:** the sidebar icon was missing. Unknown tracker totals (-1), an undefined ratio, and torrents with no size yet are now shown sensibly.
- **Wording:** "Zero KB/s" now reads "0 bytes/s" throughout.
- **Sensors:** temperatures are sorted naturally (Package, then Core 0, 1, 2 and so on), and two chips with the same name, such as two NVMe drives, are numbered.
- **Network graph:** the axis shows speeds such as "20 MB/s" instead of "3.0E7".
- **Overview:** the memory card's "used" figure now matches the percentage.
- **Start-up:** the window is brought forward at launch, and starting quietly at login finds the window by its identifier, since its title is now the server's name.

1.2.0

- **Multiple servers:** add as many Unraid servers as you like in Settings, each with its own address, API key, and SSH login. All of them are monitored and alert in the background.
- **Server picker and overview:** switch servers from the sidebar or with Cmd+1 to Cmd+9, and see every server at a glance in the all-servers overview.
- **Menu bar:** the panel lists every server, and the icon shows when any server is down.
- **Notifications** name the server when you have more than one.
- **Safer controls:** the array, reboot, and shutdown confirmations now say which server they apply to.
- **Upgrading:** an existing single server and its saved passwords are migrated automatically.
- **Deluge panel:** a Deluge tab appears when a server runs Deluge. It shows live download and upload speeds, torrent counts, free space, and every torrent with progress, speeds, ETA, ratio, and peers, with filtering, sorting, and search. You can pause and resume torrents or everything at once, force a recheck, remove torrents with or without their files, and add magnet links. The Deluge address and Web UI password are set per server in Settings.
- **Documentation:** the guides and the PDF now cover multiple servers and the Deluge panel.

1.1.0

- **Launch at login:** a new Settings option to start Unraid Watcher with your Mac. A second option starts it in the menu bar only, so alerts keep working without opening a window. It uses macOS's own login item system, so it also appears in System Settings, General, Login Items.
- **Documentation:** the guides, the troubleshooting page, and the PDF now cover launch at login.

1.0.0

First release.

- **Dashboard:** overview, system and hardware details, UPS, license and services
- **Storage:** array and disk status and temperatures, parity history, mounted filesystems, SMART health per disk with self-tests
- **Performance:** per-core CPU, memory breakdown, disk I/O, fans, CPU temperatures, network traffic, top processes, system log
- **Docker:** start, stop, restart, pause, resume, and container logs
- **VMs:** power actions, details, creation wizard, editing (basics, media, full XML), cloning, install media eject, and console access
- **Shares:** settings, per-disk usage, create, edit, and delete empty shares
- **Controls:** array start and stop, parity checks, disk spin up and down, mover, reboot and shutdown, SSH command console
- **Alerts:** macOS notifications for hot disks, hot CPU, SMART problems, and new Unraid alerts, with adjustable thresholds
- **Menu bar** status item with a quick summary
- **Security:** API key and SSH password stored in the Keychain, SSH by key or password, confirmation for destructive actions
- **Packaging:** app icon, About window, build scripts, and a DMG installer
- **License:** GNU General Public License v3.0 or later

© 2026 Ray Munro. Licensed under the GNU General Public License v3.0 or later.